Experiment 19 — Ray tracing: branch-free arrays, and a 65-element table

Mathilda — execution-speed experiments

Contents

Experiment 19 — Ray tracing: branch-free arrays, and a 65-element table

Date: 2026-07-31 · Code: `src/part.c` · Result: 1.45 s → 1.01 s, 7.4× faster than Mathematica; the sphere-parameter lookup 52.7 → 3.1 ms

Common method in `README.md`.

Hypothesis

A ray tracer written for an array language is branch-free by construction. There is no `if` anywhere: every conditional becomes a mask.

```
ok   = UnitStep[disc] UnitStep[t - eps];      (* hit, and in front of the eye *)
cand = ok t + (1 - ok) 1.*^9;                 (* miss -> +infinity *)
best = UnitStep[tmin - cand - 1.*^-6];        (* strictly closer? *)
cid  = best s + (1 - best) cid;               (* select without branching *)
tmin = MapThread[Min, {tmin, cand}];
```

That style — arithmetic on comparison results — is how all high-throughput array code expresses control flow, and the suite had never measured it end to end. It also ends with the operation experiment 12 is about: once the winning sphere per ray is known, its centre has to be gathered by index. The table has 65 entries and the index array has 262144.

The kernel

512 × 512 rays against 64 spheres on a lattice, one bounce, Lambert shading: about 2.5 × 10^8 flops over 64 passes of fifteen whole-array operations on 262144-element vectors. The scene, the camera and the light are all deterministic, so the mean pixel intensity is an exact cross-system check.

Results

Benchmark	before	after	Mathematica 14.0	NumPy 2.4.4	
Ray trace, 512² rays × 64 spheres	1.45 s	1.01 s	7.540 s	322 ms	7.44× faster than WL

Mathilda was already 5.2× faster than Mathematica before this sweep touched anything, which puts it with the Navier-Stokes row (18.4×) and the Verlet row (5.0×) in the pattern that shows up repeatedly across these eight experiments: a loop whose body is a handful of whole-array operations is where Mathilda is strongest relative to Mathematica, and where both are behind NumPy by a similar unfused-composition factor.

The finding: the source was too small to be packed

The final shading step is

```
gx = centreX[[cid + 1]];      (* 65-element table, 262144 indices *)
```

`cid + 1` is a packed `int64` array — 262144 elements, comfortably above the threshold. `centreX` is 65 elements and is, correctly, not packed.

Experiment 12 taught `Part` to read a packed index. It did not help here, because that path is only reached when the source is a buffer, and this source never will be: 65 elements is exactly the size the threshold exists to leave alone.

So the gather ran on the boxed path — one `Expr` per output element, 262144 of them, three times (x, y, z) — out of a table that is a single strided read.

gather, 262144 indices from a 65-element table	before	after
	52.7 ms	3.1 ms

The fix is the rule the third sweep named and this sweep has now applied at five sites:

> Pack the small operand up; never materialise the large one down.

A packed index list is already at or above the packing threshold, so lifting can only ever fire on a genuinely large gather, and anything `ndarray_part` declines falls straight through to the ordinary path with the original argument — so lifting cannot change an answer, an out-of-range diagnostic included.

This is the same defect as the third sweep's `Dot` case (a 32-element vector dragging a 6.4-million-element matrix onto the symbolic path) with the roles reversed: there the small operand was the second one, here it is the first, and a lookup table is small by design rather than by accident.

Where the remaining 3.2× against NumPy is

Not in the gather any more. The tracer's inner loop is 64 iterations of fifteen whole-array operations on 262144 float64 — about 32 MB of traffic per sphere, and the whole thing is memory-bound in every system. Mathilda makes fifteen passes where a fused kernel would make three or four, which is plan item 9.2, the same unfused-composition factor that appears in experiments 13, 14, 16 and 17. The per-call overhead is negligible here — 960 array operations in a 1.01 s run is 1 µs each of dispatch against ~1 ms of memory traffic.

The mask arithmetic itself is cheap and correct: `UnitStep` narrows to an exact `int64` buffer and `Times` is exact on one, so `ok t + (1 − ok) 1.*^9` stays on the buffer throughout. The one place it is not free is `da1 UnitStep[…]`-shaped mixed `float64 × int64` products, measured at ~25× a pure float multiply in NEURAL_NETWORK_TRAINING.md; the tracer does several of those per sphere and they are part of the remaining factor.

Why Mathilda is not the fastest here, and what it would take

row	Mathilda	best other	gap	cause
Ray trace, 512² × 64 spheres	1.01 s	322 ms (NumPy)	3.15×	fifteen unfused passes per sphere, at half of memory

Mathilda is 7.4× ahead of Mathematica on this row. Against NumPy, the per-operation split (printed by both source files) shows where the 3.15× is:

one pass over 262144 float64	Mathilda	NumPy	ratio
`tca = dx cx + dy cy + dz cz`	1.003 ms	0.506 ms	2.0×
`Sqrt[Clip[disc, …]]`	2.082 ms	0.565 ms	3.7×
`UnitStep[disc] UnitStep[…]`	1.423 ms	0.588 ms	2.4×
`MapThread[Min, {a, b}]`	0.812 ms	0.126 ms	6.4×
the 65-entry gather	2.401 ms	0.434 ms	5.5×

The last two are the informative ones. `MapThread[Min, …]` reads two 2 MB arrays and writes one; NumPy does it at 48 GB/s (cache-resident), Mathilda at 7.7 GB/s. Nothing about the operation is hard — it is a two-line loop over two buffers — so the shortfall is per-element dispatch inside a kernel that should be a single vectorised sweep.

The road to fastest

1. Vectorise and thread the elementwise binary kernels. `MapThread[Min]` at 6.4× NumPy on a cache-resident array is the clearest single number in this experiment. `nd_mapthread2`'s inner loop is a scalar `if` per element; `fmin`/`fmax` intrinsics vectorise, and `nd_parallel_for` is already used by the unary kernels. Expected: 0.812 ms → ~0.15 ms.
2. Fuse the mask arithmetic (plan 9.2). `ok tt + (1 − ok) 1e9` is four passes to compute a select; `bt s + (1 − bt) cid` is four more. A fused `where`-style kernel — or simply recognising `a x + (1 − a) y` — turns eight passes into two, per sphere, 64 times.
3. Thread the gather (experiment 12's roadmap, item 2). 2.401 ms for 262144 random reads out of a 520-byte table is entirely latency, which threads perfectly.
4. Mixed **`float64 × int64`** elementwise. `ok tt` and `bt s` multiply a float64 array by an `int64` mask; that pair goes through the generic accessor rather than a widening loop, and is worth ~25× on those lines (measured in experiment 17).

Items 1–4 are all inner-loop kernel work with no design question attached, and together they are the whole of the 3.15×. This is the most straightforwardly closable gap in the sweep.

Verification

- The mean pixel intensity agrees to full printed precision across Mathilda, Mathematica and NumPy.
- `test_fifth_sweep_fast_paths` covers the lifted gather differentially: real and integer sources, a `Rational` source that cannot pack, a symbolic source, an out-of-range index, and the composition (`Ramp` of a shading term times a gathered table) that the tracer actually ends with — each the same source with automatic packing on and off.